IIIT Bhubaneswar

Web Search Mining (WSM): Assignment Report

A Mini Information Retrieval System for Shakespeare's Plays

Name	ID	Branch	Institute
Anish Ray	B524011	CE	IIIT Bhubaneswar

1. Introduction

This project is a small search engine in Python. It indexes 25 text documents, the five acts of five Shakespeare plays (*All's Well That Ends Well*, *Antony and Cleopatra*, *As You Like It*, *Hamlet*, *Romeo and Juliet*, about 131,700 words from the Folger Shakespeare Library). A user types a query and gets a ranked list of acts with a snippet around the match. I built every stage myself using classical retrieval techniques only: no machine learning, embeddings or search libraries.

Problem and motivation: scanning files for a word is slow and does not say which file matches best. A search engine builds an index once and scores documents, and I wanted to understand each stage by implementing it. **Objectives:** (1) inverted index; (2) Boolean, wildcard, phrase and proximity queries; (3) TF-IDF and cosine ranking with snippets; (4) stemming, sublinear TF and Rocchio relevance feedback; (5) measure ranking quality with Precision, Recall, F1, MAP and MRR; (6) a simple interface.

2. Technologies and Tools

Technology	Used for	Why
Python 3.12 (standard library only: `re`, `math`, `collections`, `bisect`, `json`, `argparse`)	All retrieval logic and the evaluation script	Dicts, sets and `Counter` map directly onto an inverted index; no dependencies (`requirements.txt` is empty).
`http.server` + HTML/CSS/JavaScript	Web interface (`server.py`, `web/index.html`)	A browser page looks better than a desktop window and needs no framework.
Tkinter	Original desktop GUI (`gui.py`, still available with `--tk`)	Ships with Python.
Git / GitHub	Version control	The commit history records the order of development.

3. Architecture

Retrieval logic is separate from the interface: the GUI and the web server only call `SearchEngine.search()` and `SearchEngine.feedback()`. This let me replace the Tkinter window with a web page without touching the search code.

```
.txt files -> document_loader -> text_processor -> indexer -> (index built once at start-up)
                              (tokenize, stop words,   (postings, term counts,
                               optional stemming)       positions)
query -> query_processor (+ wildcard) -> ranker (TF-IDF, cosine, Rocchio) -> search_engine
         AND/OR/NOT, phrase, proximity                                       (snippets)
                                                                            /          \
                                                                      server.py        gui.py
                                                                  (web/index.html)    (Tkinter)
```

Module	Responsibility
`document_loader`, `text_processor`, `stemmer`	Load files; one `preprocess()` shared by documents and queries; Porter stemmer written from the published algorithm.
`indexer`	Posting lists, term frequencies, document lengths, positional index; phrase and proximity matching.
`query_processor`, `wildcard`	Parse the query; combine posting lists; expand wildcards through regular expressions on the vocabulary.

Module	Responsibility
`ranker`	TF-IDF weights, cosine similarity, Rocchio query expansion.
`search_engine`	Wires the parts together, builds snippets; the only class the interfaces use.
`evaluation`, `evaluate.py`	Metrics, and an experiment script over 13 hand-judged queries.

4. Development Methodology and Implementation

Stage (date)	What I built
1. Core pipeline (2 Sep)	Corpus, loader, text processing, inverted index, AND + wildcard queries, TF-IDF cosine ranking, `SearchEngine`, Tkinter GUI, README.
2. Usability (2 Sep)	Query-focused snippets; OR and NOT; syntax hint under the search box.
3. Ranking fixes (3 Sep)	Common-word and wildcard weighting fixes (Section 7); GUI restyle.
4. Extensions (5 Nov)	Stemming, sublinear TF, phrase queries, Rocchio feedback, evaluation script, web interface, proximity queries.

Index. `build()` makes one pass per document and stores posting lists (`term -> set of doc IDs`), a `Counter` of term frequencies and the document length. For this corpus: 9,678 distinct terms from 75,150 indexed tokens (6,725 terms with stemming).

Ranking. Documents and queries are TF-IDF vectors; the score is their cosine. `idf = log(N/df) + 1`. Document vector lengths are computed once at start-up. Sublinear TF replaces `tf` by `1 + log(tf)`.

```
for term, q_weight in query_vector.items():            # cosine for one candidate document
    tf = self.index.term_frequency(term, doc_id)
    if tf > 0:
        dot += q_weight * (self._tf_weight(tf) * self.idf[term])
score = dot / (query_norm * self.doc_norm[doc_id])
```

Query processing. `OR` turns the positive terms from intersection into union; `NOT` or `-` excludes the next term. A wildcard becomes a regular expression matched against the vocabulary, and the union of the matching terms' posting lists is used. Matched terms are grouped per query term and each group gets one unit of term frequency, so a wildcard cannot outweigh plain words.

Phrases and proximity. The positional index stores `term -> {doc: [positions]}` over the token stream *with* stop words kept, so a phrase made of stop words ("to be or not to be") still works. A phrase keeps start position `p` only if word `i` occurs at `p + i`. For `a /N b`, each position `p` of `a` is binary-searched against the sorted positions of `b` for a hit in `[p - N, p + N]`.

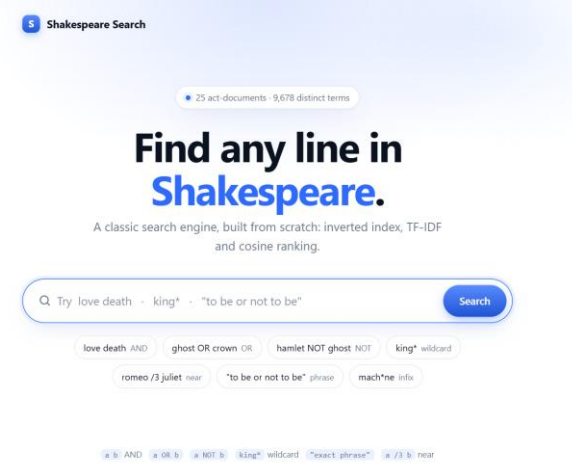
```
lo = bisect_left(b_positions, p - distance)       # proximity: b within N positions of a
hi = bisect_right(b_positions, p + distance)
pairs += max(hi - lo - (1 if term_a == term_b else 0), 0)
```

Rocchio feedback. `q' = α·q + β·mean(relevant) − γ·mean(non-relevant)` with $\alpha = 1$, $\beta = 0.75$, $\gamma = 0.15$; document vectors are length-normalized, negative weights dropped, and only the 25 heaviest feedback terms are added. The expanded query is scored against all documents. **Snippets** are about 200 characters centred on the first matched word, phrase or window.

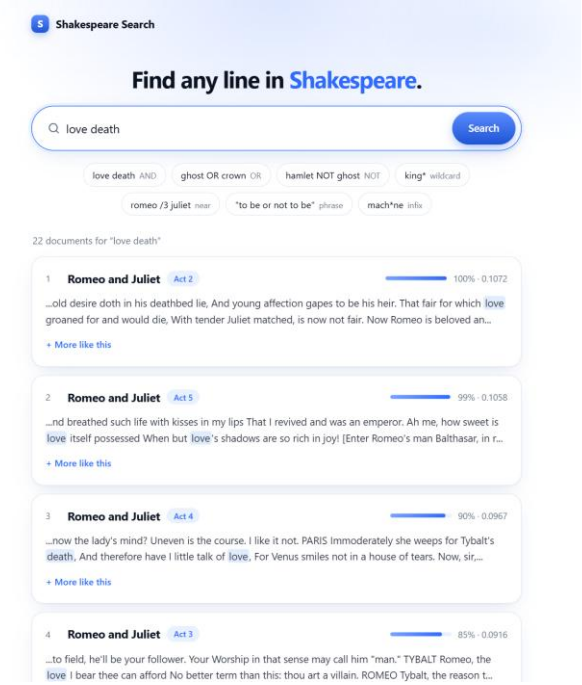
5. Features and Functionality

Feature	Behaviour	Example
Boolean AND / OR / NOT	AND by default; one `OR` makes all terms optional; `NOT` or `-` excludes. No parentheses.	`love death`, `love OR death`, `hamlet NOT ghost`
Wildcards	* = zero or more characters, at the start, middle or end.	`king*`, `*ing`, `mach*ne`
TF-IDF ranking	Cosine ranking; a relative score (% of best) is shown as a display aid.	any query
Phrase queries	Words in quotes must be adjacent and in order; can be negated.	`"alas poor yorick"`
Proximity queries	Two plain words within N positions, either order.	`romeo /3 juliet`

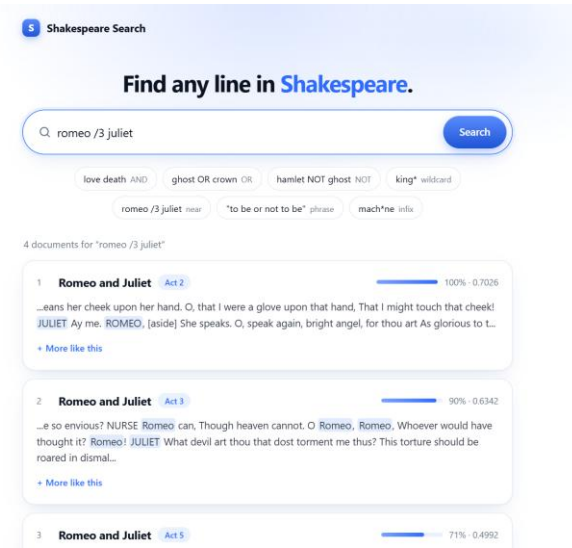
Feature	Behaviour	Example
Stemming, log TF	Optional flags `--stem`, `--log-tf`.	`loving` also finds *love*
Relevance feedback	"+ More like this" under a result re-ranks with Rocchio and lists added terms.	`ghost`, then mark Hamlet Act 1
Messages	Empty query, only stop words, only exclusions, no match: a message, never a crash.	`machine learning`
Interfaces	Web page (`python main.py`); Tkinter window (`--tk`). The page is responsive (works at phone width) and supports dark mode.	Figure 1



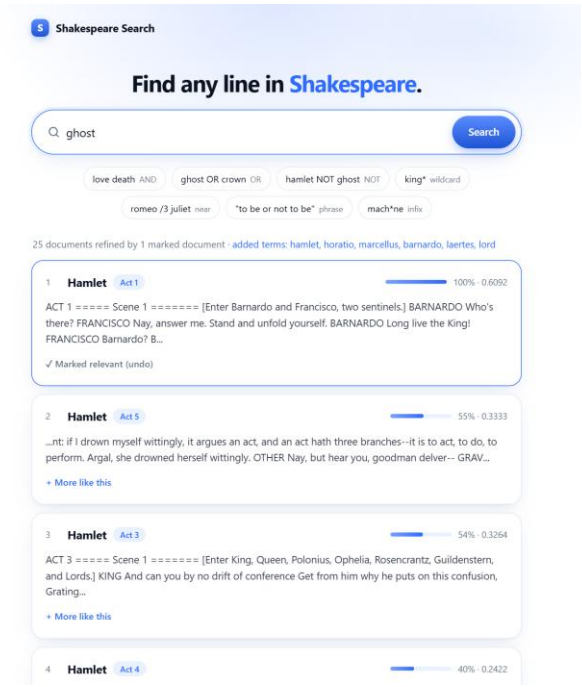
(a) Home page



(b) AND query: love death



(c) Proximity query: romeo /3 juliet



(d) Feedback on ghost, after marking Hamlet Act 1

Figure 1: Screenshots of the web interface. A Tkinter screenshot can be added by running python main.py --tk.

6. Testing and Results

I tested by running queries through the engine and the web page and checking results against the text. There are no automated unit tests. Ranking quality was measured with `evaluate.py`. Scores below are cosine values (raw TF, no stemming).

Query	Expected	Actual
`love death`	Acts with both words	22 docs; top Romeo and Juliet Act 2 (0.107)
`hamlet NOT ghost`	Hamlet acts without *ghost*	3 docs: Hamlet Act 5 (0.685), Act 2, Act 4
`king*` / `mach*ne`	King forms / *machine*	18 docs, top Hamlet Act 4 / 1 doc, Hamlet Act 2
`"to be or not to be"`	Hamlet's soliloquy	1 doc: Hamlet Act 3
`romeo /3 juliet`	Names close together	4 docs; top Romeo and Juliet Act 2 (0.703)
`the and of`; `machine learning`; `-death`	Handled messages	`only_stopwords`; `no_match`; `no_positive_terms`
`loving`, stemming off vs on	Stemming widens matches	Top result changes from As You Like It Act 2 to Act 4
`ghost` + mark Hamlet Act 1	Query expands towards Hamlet	Added: hamlet, horatio, marcellus, barnardo, laertes, lord

Ranking quality. I wrote 13 queries (11 keyword queries joined with OR, 2 phrase queries) with the acts I judged relevant (`data/relevance.json`). The judgments are mine, at act level and few, so differences of a few hundredths are noise. Results at k = 3:

Configuration	P@3	R@3	F1@3	MAP	MRR
raw TF	0.692	0.867	0.733	0.921	1.000
log TF	0.718	0.905	0.764	0.932	0.962
raw TF + stemming	0.692	0.867	0.733	0.905	0.962
log TF + stemming	0.692	0.867	0.733	0.923	0.962

Log TF helped slightly; stemming helped some queries and hurt others. **Rocchio** was tested on the residual collection (the judged document removed before scoring): over 11 queries MAP fell from 0.775 to 0.674. The added terms are mostly character names, which lift every act of the same play. Smaller β (0.1 to 0.2) came out near the baseline, but I chose that after seeing the test set, so I kept the textbook values.

7. Challenges and Solutions

Problem	Solution
A one-word query for a very common word gave no ranking: `log(N/df)` is 0 for a word in every document.	Used `log(N/df) + 1`.
A wildcard such as `king*` expanded to many words and outweighed the plain words in the query.	Group matched terms per query term and give each group one unit of term frequency.
A phrase of stop words cannot be found if stop words are removed before positions are recorded.	Built the positional index with stop words kept.
Stemming vs. snippets and wildcards: raw text keeps original word forms, and wildcard patterns match the stemmed vocabulary.	Snippets stem each raw word before comparing. Wildcards are not stemmed, so `*ing` finds little when stemming is on (documented limitation).
A stray `death /3` was read as the word "3" and matched "ACT 3" headings.	The parser now ignores an unpaired `/N`.
The web page highlighted *deathbed* for the query *death*, but the engine matches whole words.	The highlighter now matches whole words; only wildcards match prefixes.

8. Conclusion

I built a working search engine on the standard library alone. It supports Boolean, wildcard, phrase and proximity queries, TF-IDF cosine ranking, snippets, optional stemming and log TF, and Rocchio feedback, and

I measured its ranking quality instead of judging by eye. Stemming and Rocchio did not improve results on this small collection, and I report that as measured. I learned how the index structures and the weighting details (such as the smoothed IDF) shape the ranking.

Run: `python main.py` (web, http://127.0.0.1:8000), `--tk` (Tkinter), `--stem`, `--log-tf`; `python evaluate.py` for the experiments. **Source:** https://github.com/An1sh-R/WSM-project